

SUPERVISED MACHINE LEARNING: CLASSIFICATION

MODULE 5: ENSEMBLE MODELS

TABLE OF CONTENTS

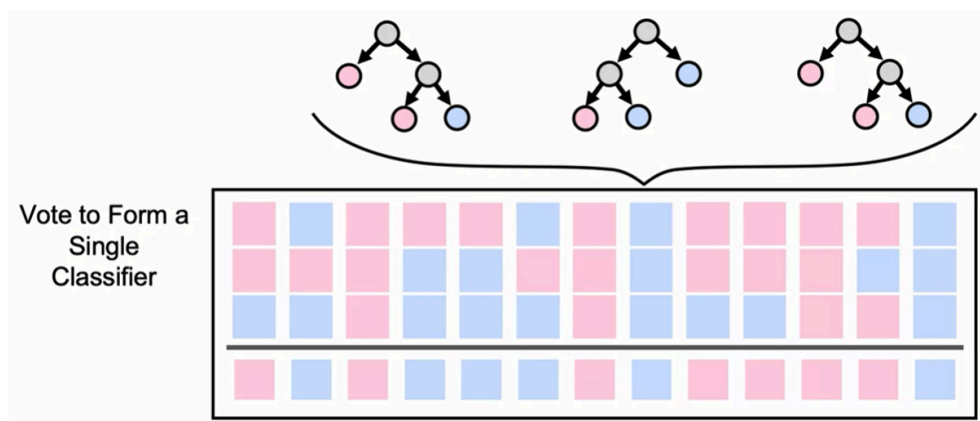
Concept.....	2
Bagging (Bootstrap Aggregating).....	2
Random Forest.....	3
Extra Random Trees.....	3
Boosting.....	4
Stacking.....	5
Key Takeaways.....	6

Concept



- Ensemble methods combine multiple models (learners) to improve accuracy and generalization.
- Main types:
 - Bagging (Bootstrap Aggregating)
 - Random Forests
 - Boosting (AdaBoost, Gradient Boosting)
 - Stacking

Bagging (Bootstrap Aggregating)



- Creates many trees using **bootstrapped samples** (random subsets with replacement).
- Each tree votes → majority vote = final prediction (**meta-classifier**).
- Reduces **variance** and limits **overfitting**.
- Trees can be built **in parallel** → efficient computation.

```
from sklearn.ensemble import BaggingClassifier
model = BaggingClassifier(n_estimators=50)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Random Forest

- Extension of bagging: adds randomness in features.
- Each tree uses a random subset of features (thuộc tính) for splitting.
- Reduces correlation between trees → lower variance, higher stability.
- Use cross-validation and out-of-bag error to find optimal number of trees.

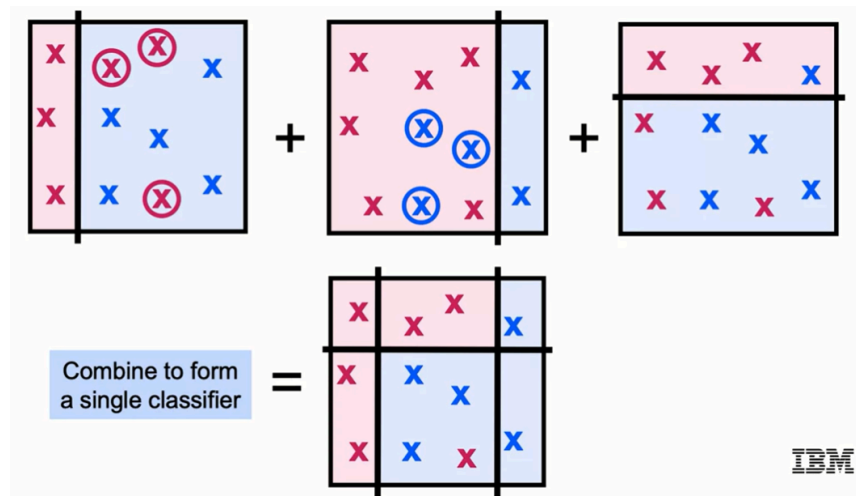
```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=50,
max_features='sqrt')
rf.fit(X_train, y_train)
```

Extra Random Trees

- Similar to Random Forest but introduces extra randomness:
 - Splits chosen completely at random, not via greedy search.
- Each individual tree is weaker, but combined → strong ensemble.

```
from sklearn.ensemble import ExtraTreesClassifier
et = ExtraTreesClassifier(n_estimators=100)
et.fit(X_train, y_train)
```

Boosting



- Sequentially builds trees, where each **new tree focuses on correcting previous errors**.
- Trees are **dependent**, not parallel.
- Each weak learner (decision stump – cây gốc nông) gets a **weight** based on performance.
- Can overfit if not tuned properly.

Algorithm	Loss Function	Sensitivity
0-1 Loss	Theoretical, hard to optimize	No
AdaBoost	Exponential loss	Sensitive to outliers
Gradient Boosting	Log-likelihood (deviance)	Robust to outliers

- Main Hyperparameters
 - learning rate (tốc độ học)
 - n_estimators (số lượng cây)
 - subsample
 - max_features

```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(learning_rate=0.1,
n_estimators=200, subsample=0.5)
gb.fit(X_train, y_train)
```

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
ab = AdaBoostClassifier(
base_estimator=DecisionTreeClassifier(max_depth=1),
learning_rate=0.1,
n_estimators=200)
ab.fit(X_train, y_train)
```

Stacking

- Combines predictions of **different models (heterogeneous models)** such as logistic regression, SVM, and random forest.
- Each model's output becomes a **meta-feature**; a **meta-classifier** learns from these.
- Final decision: via **voting** (majority or weighted) or another model.

```
from sklearn.ensemble import VotingClassifier
vc = VotingClassifier(estimators=[
    ('lr', LogisticRegression()),
    ('rf', RandomForestClassifier()),
    ('svm', SVC(probability=True))
], voting='soft')
vc.fit(X_train, y_train)
```

Key Takeaways

- **Bagging** → reduces **variance** by parallel averaging.
- **Random Forest** → adds randomness in features to decorrelate trees.
- **Boosting** → reduces **bias**, sequentially learns from residuals.
- **Gradient Boosting** → robust version using log-loss.
- **Stacking** → generalizes bagging by mixing any models, not just trees.
- **Overfitting** is common in boosting and stacking → use **cross-validation** and **hyperparameter tuning**.